

# **SE4020 - Mobile Application Development & Design - Assignment 2**

Part A - iPadOS or macOS App

## Design & Implementation Documentation

**Submitted by:**

IT22346322 - MJM ARSHAQ

**Platform:** iPadOS (landscape, multicolumn)

**User role:** Keyworker (Early Years Practitioner)

## 1. Overview

NurseryConnect helps a single nursery **Keyworker** ("Sarah Mitchell") turn their **daily diary logs** (activities, meals, naps, nappies, wellbeing) into **development & wellbeing insights**, and file **safeguarding incident reports all** on an iPad sized for on the floor, multicolumn work. Per the brief, there is **no login or access control**: the app opens straight into its main functionality. The architecture is unchanged MVVM + Services from Assignment 1 (Views → ViewModels → Services → SwiftData), replatformed from an iPhone TabView to an iPad NavigationSplitView.

## 2. Extension justification what's new vs Assignment 1, and why

The brief asks for a meaningful platform extension, not a reskin. Three additions were chosen because each exploits something the iPad does that the A1 iPhone build could not:

| Extension                                                                     | Why it earns its place                                                                                                                                                                                                               | Maps to rubric                        |
|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| <b>Multicolumn NavigationSplitView</b><br>(sidebar → roster → child day view) | The keyworker constantly <i>switches between children while keeping the roster in view</i> the iPad's width turns a threetap iPhone drilldown into a single persistent context. This is the structural backbone, not the decoration. | Criterion 3 platform specific feature |
| <b>Development &amp; Wellbeing Insights dashboard (Swift Charts)</b>          | The keyworker already logs diary data daily; the genuinely <i>new</i> value is reading it back as <b>trends</b> (mood line, nap bars, meal portions) to spot wellbeing changes early an EYFS "ongoing assessment"                    | Criterion 4 advanced library          |

|                                                             |                                                                                                                                                              |                                       |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
|                                                             | duty. Reuses existing data, adds an advanced library.                                                                                                        |                                       |
| <b>PencilKit handwritten signature</b> on the incident form | A safeguarding record needs a <b>human attestation</b> ; Apple Pencil signoff is the natural iPad analogue of a paper signature and anchors the audit trail. | Criterion 3 platform specific feature |

**Deliberately deferred:** a PDFKit "signed parent report" was scoped out to keep the build focused and verifiable within the timebox noted honestly rather than half built.

**Why the Keyworker role (carried from Assignment 1):** under EYFS 2024 every child has an assigned keyworker; they are the single most active user in the case study, which makes a no login, single user app both realistic and brief compliant.

### 3. Design choices

#### 3.1 Visual language: "calm, professional childcare"

The palette is a deliberate rejection of stock iOS blue and of a toy/childish aesthetic:

| Token             | Light   | Role                                                      |
|-------------------|---------|-----------------------------------------------------------|
| Brand blue        | #3D7DCA | Primary actions, selection, accents                       |
| Brand soft        | #E8F1FB | Selected/highlighted row backing                          |
| App background    | #F7F6F2 | Warm offwhite never pure white                            |
| Surface           | #FFFFFF | Cards                                                     |
| <b>Danger red</b> | #C8443B | <b>Reserved exclusively for incidents &amp; allergies</b> |
| Success green     | #3E9D6B | Saved/dispatched, "most/all eaten"                        |
| Warning amber     | #CC8A2E | Dietary notes, pending                                    |

The single most important design rule is **red is a reserved semantic**: it never appears as decoration, only as severity (an allergy chip, the "Report Incident" button, a "No photos" consent flag). This makes the interface *teach* the user a red element always means "pay attention to safety." Every token is an asset catalog Color Set with explicit light **and** dark variants, so Dark Mode is correct for free (criterion 7).

#### 3.2 Layout cards on a warm canvas

White rounded cards float on the offwhite background (cardStyle()), giving a calm, clinical but warm separation of concerns. Typography rides Dynamic Type text styles (AppTypography) so the app respects accessibility sizing, and spacing is centralized (AppSpacing) so touch targets stay  $\geq 44\text{pt}$  for arm's length use on the nursery floor.

### 3.3 Information hierarchy reflects real frequency of use

The most frequent action (quick logging a diary entry) sits at the **top** of the child day view in a QuickActionsBar; the new Insights dashboard, though it's the headline *feature*, lives one deliberate navigation step away in the sidebar because the keyworker logs many times a day but reviews trends occasionally. The UI is ordered by *how often* the keyworker does something, not by what is technically newest.

## 4. Implementation decisions

### 4.1 Architecture MVVM + Services over SwiftData

Views → ViewModels → Services → SwiftData

- **Models** (Child, DiaryEntry, IncidentReport) are pure @Model types stored properties + simple computed helpers only, with cascade delete relationships.
- **Services** are @MainActor structs exposing async throws I/O and typed Localized Error enums (DiaryService, IncidentService, InsightsService, ChildRosterService, MockDispatchService). Keeping persistence and validation here keeps views dumb and logic testable.
- **ViewModels** are @Observable @MainActor final classes with explicit state enums (.idle/.loading/.loaded/.failed), consumed via @Bindable.
- **One ModelContainerProvider** offers a production on disk container and an in memory one for previews/tests; SeedDataService seeds 6 children + 7 days of diary data + a sample incident so **every screen is nonempty on first launch**.

### 4.2 Why this layering paid off

Because aggregation logic lives in a pure, injectable InsightsService (calendar/clock injected), the charts were **unit testable without a UI** 24 Swift Testing tests run against an inmemory container, including the date window aging behavior that caused a real bug.

### 4.3 Platform features, implemented carefully

- **NavigationSplitView** (RootSplitView): column widths are *not* hardcoded; selection flows through the view models, so it scales across iPad sizes and Stage Manager.
- **PencilKit** (SignatureCaptureView): a PKCanvasView wrapped in UIViewRepresentable, set to .anyInput so a finger signs on the simulator; strokes flatten to a PNG stored as @Attribute(.externalStorage) on the incident. The incident form's isValid *requires* a signature you cannot file an unsigned record.
- **Hardware keyboard bonus:** ⌘N (new diary entry for the visible child) and ⌘F (focus roster search), because iPad users often have a Magic Keyboard.
- **UIKit isolation:** all PencilKit/UIKit code is #if canImport(Uikit) guarded so the macOS SourceKit index stays quiet (the same pattern as Haptics).

## 5. Regulatory awareness

Childcare software is safeguarding software; several decisions are driven by regulation, not aesthetics:

| Regulation / standard                                        | How the app reflects it                                                                                                                                                                                                                                       |
|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>EYFS 2024 assigned keyworker &amp; ongoing assessment</b> | The whole role model (one keyworker, their assigned children) and the Insights dashboard (reading diary trends to monitor development/wellbeing) operationalise the keyworker's statutory duty.                                                               |
| <b>RIDDOR inspired incident order</b>                        | The incident form sections follow a defensible field order: category → severity → when/where → what happened → body map → action taken → witnesses → signoff.                                                                                                 |
| <b>GDPR accuracy / no backdating</b>                         | The incident timestamp is <b>readonly</b> , taken from the system clock, with a visible footer explaining it "cannot be edited" preventing retroactive falsification of a safeguarding record.                                                                |
| <b>GDPR data minimization / consent</b>                      | A <b>photography consent</b> flag surfaces as a red "No photos" capsule on the child header; the roster service filters per a GDPR rule (covered by a unit test).                                                                                             |
| <b>Safeguarding immutability &amp; attestation</b>           | Incident reports are <b>immutable for post submission</b> , and a <b>PencilKit signature is mandatory for the</b> human anchor of the audit trail. Allergies are rendered in reserved red on both roster and detail, so a life critical fact is never buried. |

These are documented in code comments at the point of decision (e.g. the readonly timestamp footer, the allergy color rule), so the *why* travels with the *what*.



## 6. Challenges faced (and how they were resolved)

These are the moments the build genuinely fought back each was diagnosed with real evidence (compiler output, a crash backtrace, a simulator screenshot, or the SQLite store), not by trusting generated code. Full detail is in [aiusagelog.md §3](#).

- "Charts showing nothing" is the hardest bug.** The Swift Charts dashboard rendered empty against an existing store. By querying the SwiftData SQLite file directly, I confirmed the *data existed* but had aged out of a 7day window anchored to "today."  
**Fix:** anchor the window to the **latest entry date**, not the wall clock proven by backdating the store for 30 days and rescreenshotting. This is exactly why aggregation lives in a testable service with an injectable calendar.
- UIColor vs asset colors.** The first design system drafts used UIColor, which raised false "cannot find in scope" errors because SourceKit indexes against macOS (no UIKit).  
**Fix:** asset catalog Color Sets referenced as Color("BrandBlue") which also bought Dark Mode for free.
- Quick log buttons clipping in portrait.** A View That Fits silently fell back to a horizontal scroll view in the narrower portrait column, clipping the last button.  
**Fix:** equal width flexible buttons that always fit.
- Selection contrast bug.** On the brand blue selected row, subtitle text and the child avatar were disappearing.  
**Fix:** selection of aware styling subtitles using semantics. secondary (inverts to readable white on highlight) and the avatar gets a solid white backing when selected.
- SwiftData test crash.** The first generated test suite crashed with EXC\_BREAKPOINT because Swift Testing runs tests concurrently, and the in-memory containers raced on teardown.  
**Fix:** @Suite (. serialized) + container retention diagnosed with the actual crash

backtrace.

## Tool

## 7. The AI driven design process

### Code generation AI Claude Code

Full phase by phase detail, including every correction, in summary, AI acted as a **pair programmer and reviewer**, never an unchecked generator:

- **Used for:** scaffolding the design system and NavigationSplitView shell, reimplementing the A1 models/services/view models, the PencilKit signature pad, the Swift Charts InsightsService/dashboard, accessibility polish, and authoring the test suite + fixtures.
- **Verified by** every phase ended in a clean xcodebuild, a simulator run screenshotted, and xcodebuild test → TEST SUCCEEDED. Every generated file was read for correctness before being kept.
- **Caught and corrected:** the six issues above each a case where AI output was wrong, and I found it through real evidence, demonstrating AI was *checked, not trusted*.

### 1. Tools used

|                                          | Role                                                                                                             |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Claude Code (Anthropic, Opus 4.x)</b> | In terminal pair programmer: scaffolding, SwiftUI/SwiftData implementation, refactors, test authoring, debugging |
| <b>Xcode 26 + iPad (A16) simulator</b>   | Authoritative build/run/test environment every AI suggestion was compiled & run here                             |
| <b>AI UI mockup tools</b>                | Early-stage layout/visual exploration for the iPad split view (design ideation only)                             |

### 2. How AI was used, phase by phase

- **Used for:** generating the designsystem tokens (AppColors, AppTypography, AppSpacing, AppIcons, ViewStyles, Haptics), the ModelContainerProvider factories, and the threecolumn NavigationSplitView shell.

- **Verified by** a clean xcodebuild and the simulator confirming the branded shell appears.
- **Rejected/corrected:** the AI's first colour approach used UIColor, which raised false errors because SourceKit indexes against macOS (no UIKit). I had it switch to **assetcatalog Color Sets** referenced as Color("BrandBlue").

#### Reimplementing the Assignment 1 core

- **Used for:** the full Child / DiaryEntry / IncidentReport @Model types, the @MainActor services with typed LocalizedError enums, the MVVM view models, and the diary/incident UI.
- **Verified by:** CLI build + manual walkthrough of roster → day view → diary sheet → incident form → history.
- **Corrected:** A DiaryEntry initializer argumentorder bug and a viewmodel init that failed to store its service both caught by reading the compiler output, not by trusting the generated code.

#### PencilKit native feature

- **Used for:** the PKCanvasView UIViewRepresentable signature pad, persisting the flattened PNG to @Attribute(.externalStorage) on IncidentReport.
- **Verified by** building green and reasoning about the #if canImport(Uikit) guards (the same pattern as Haptics) so the macOS index stays quiet.

#### Swift Charts dashboard (advanced library)

- **Used for:** InsightsService aggregation, InsightsViewModel, and the InsightsDashboardView (mood LineMark, nap/meal BarMark).
- **Verified by** installing the simulator and **screenshotting** the rendered charts, this is how the "nothing showing as graph" issue was caught.

#### Polish & accessibility

- **Used for:** selectionaware row styling, the heartlogo wordmark, VoiceOver labels on chart marks, and the quicklog layout fix.
- **Verified by** simulator screenshots of the selectedrow contrast, the heart logo (landscape), and the quicklog row fitting the portrait column.

#### Testing

- **Used for:** authoring 24 Swift Testing unit tests + 4 XCUITest happy paths and the in-memory Test Support fixtures.
- **Verified by:** xcodebuild test → TEST SUCCEEDED; AI introduced test flakiness was diagnosed from a real crash log and fixed.



## Verification methodology

For every phase:

1. **Compile** - xcodebuild build destination 'platform=iOS Simulator, name=iPad (A16)' was treated as the source of truth (the live editor index lags and shows false "Cannot find X in scope" errors).
2. **Run installed** and launched on the simulator; inspected screenshots.
3. **Test** added automated xcodebuild test coverage.
4. **Read every** generated file was read for correctness (compliance comments, @MainActor isolation, SwiftData #Predicate rules) before being kept.